



Greedy Algorithms 2

- Priyansh Agarwal

Activity Selection Problem: [Link](#)

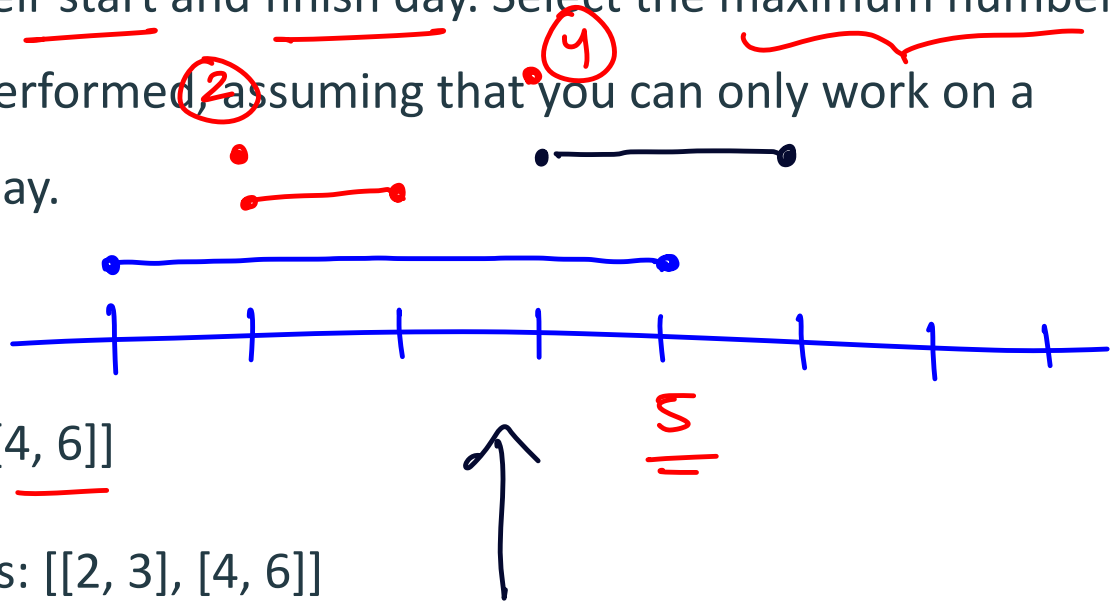


Given N activities with their start and finish day. Select the maximum number of activities that can be performed, assuming that you can only work on a single activity at a given day.

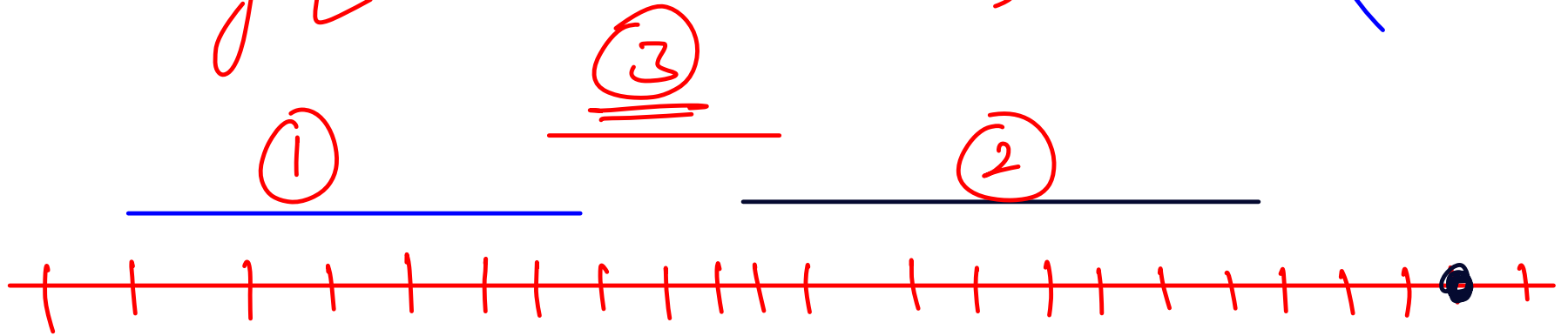
Example:

Activities = [[1, 5], [2, 3], [4, 6]]

Ans = 2, selected activities: [[2, 3], [4, 6]]

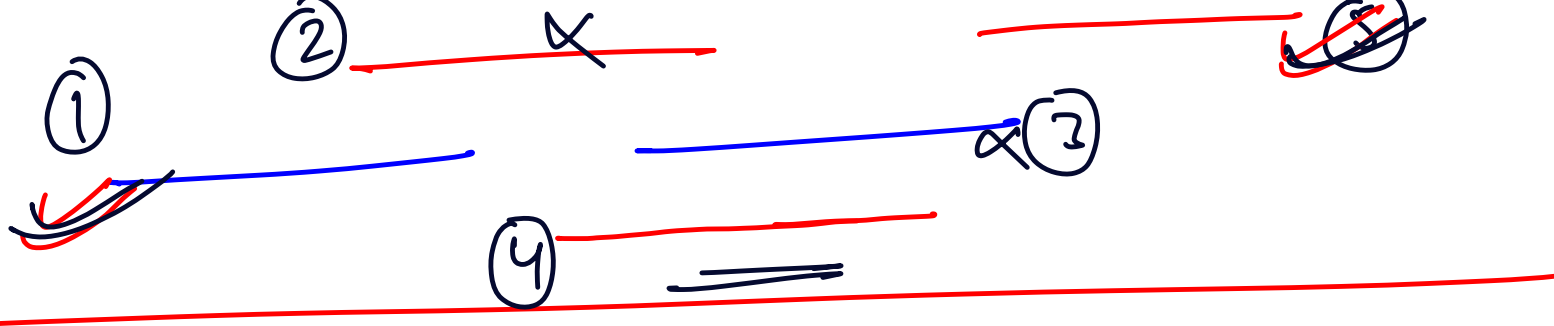


✓ Claim $\rightarrow$ Do the activities in the order of
length of activity $[\text{end Time} - \text{start Time}]$ α



α Claim $\rightarrow$ Choose intervals in the order of
how of how many intersections they have
(ascending order)





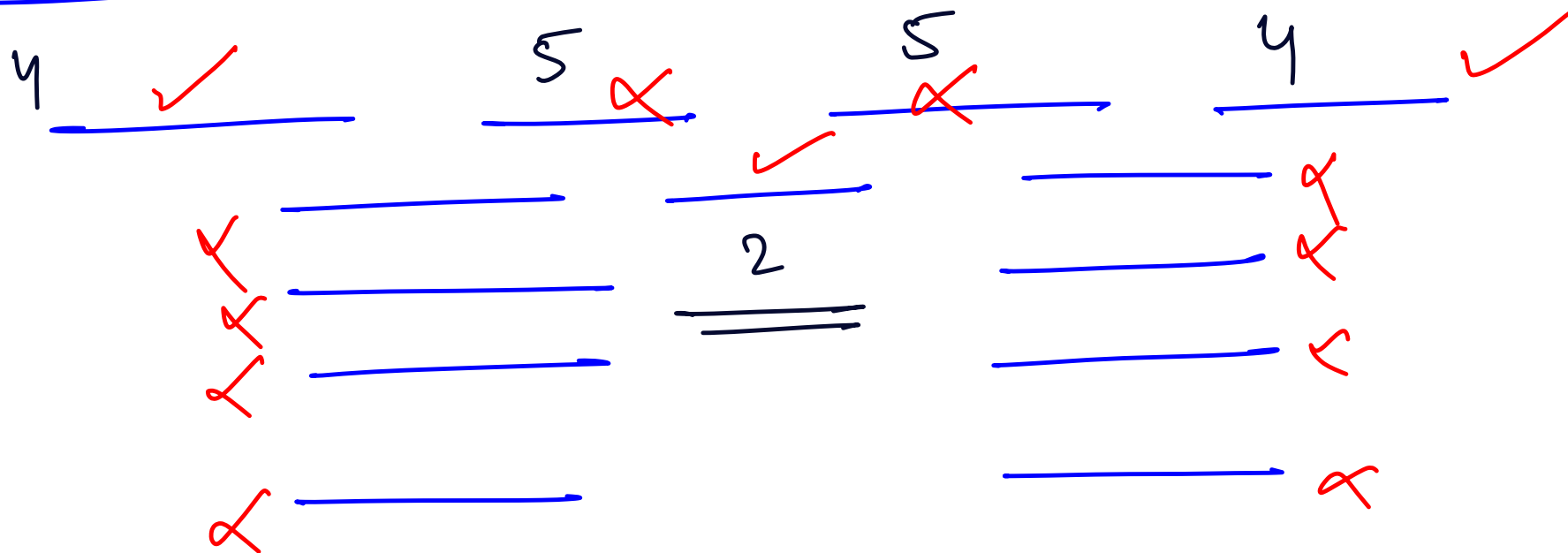
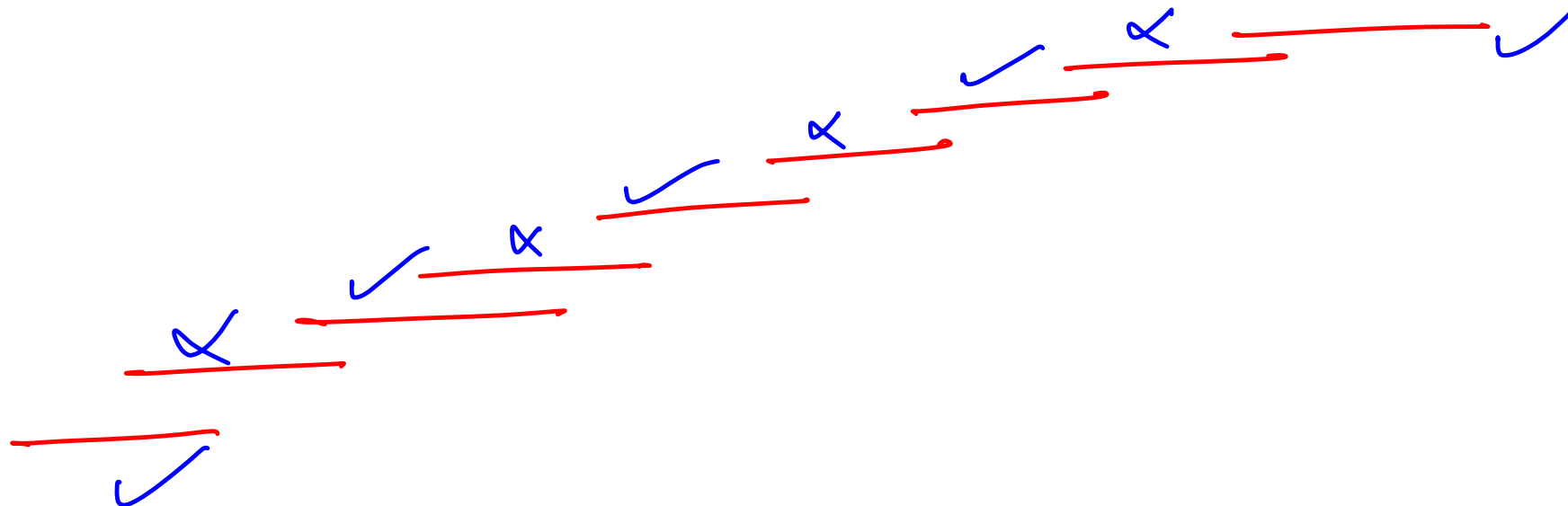
n

→ how to find out the no. of intersections
of each interval in the remaining set

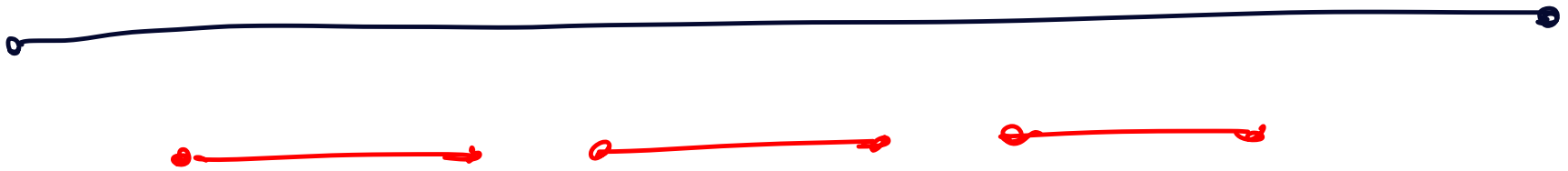
- some sort of sorting → $O(n \log n)$
- $O(n)$

$$\underline{\underline{O(n^2 \log n)}}$$

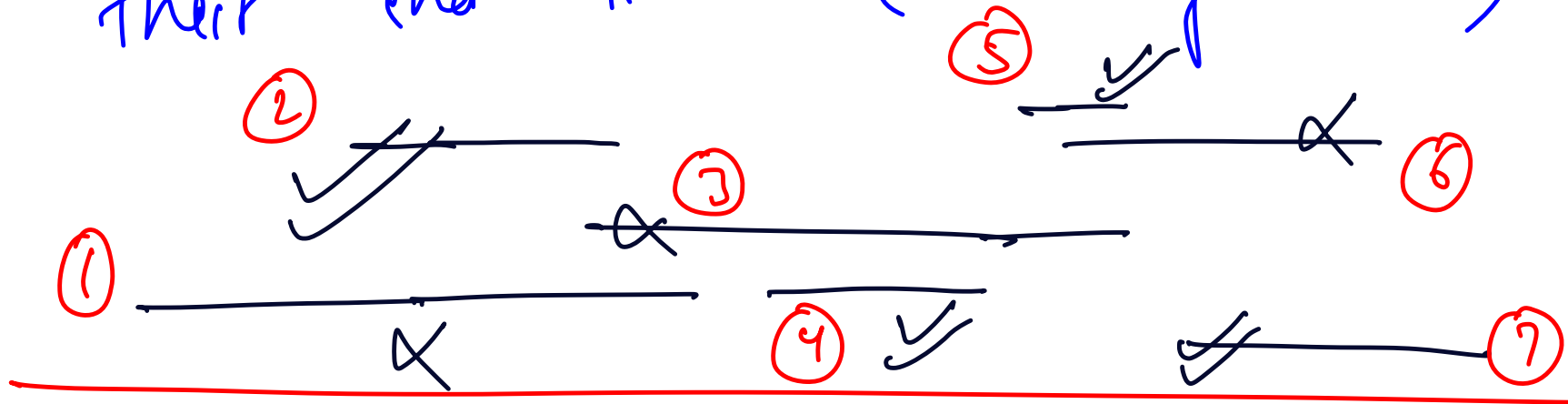
DP → $O(n^2)$



Claim ③ : Choose intervals in the order of their start time (ascending order)



Claim ④ Choose intervals in the order of their end times (ascending order)



When going from left to right, picking a segment which ends first, ensures that we will be able to pick up more segments in future

Proof

① Which segment to choose first

 $GA =$ segment which ends first
 $OA =$ segment which doesn't end first.

$$GA = \frac{1}{1+x}$$

$$OA = \frac{1}{1+y}$$

Assume $OA > GA$

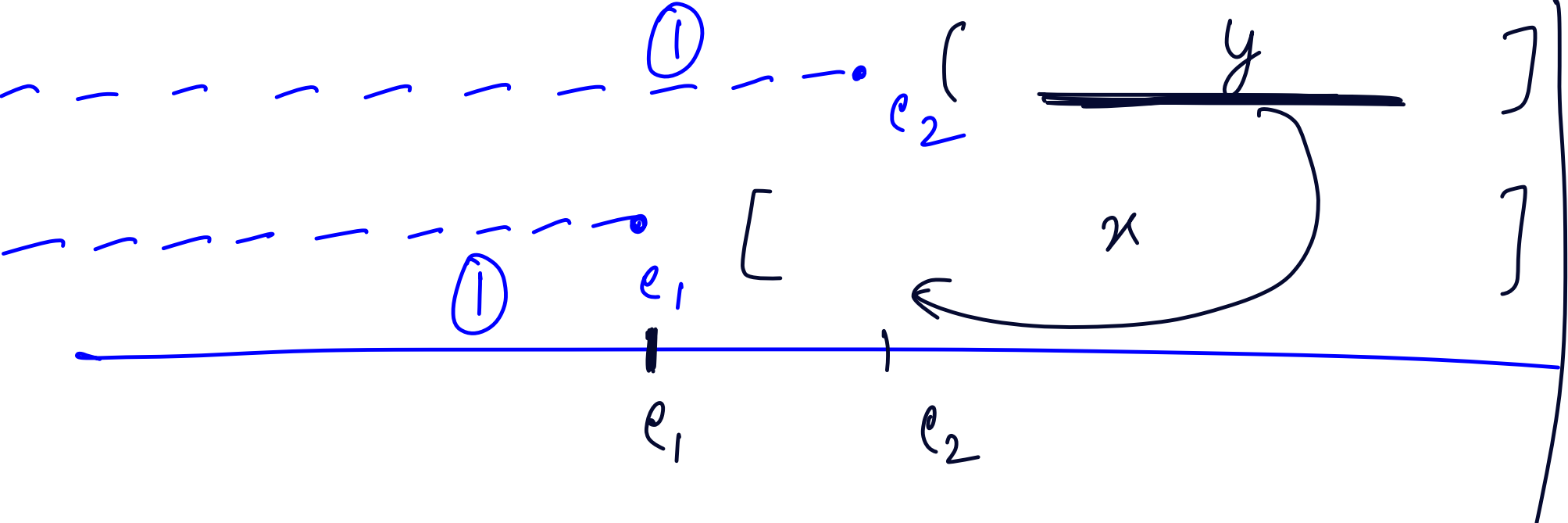
$$1+y > 1+x$$

$$y > x$$

$$GA \rightarrow [s_1, e_1]$$

$$OA \rightarrow [s_2, e_2]$$

$$e_2 > e_1$$



therefore $x < y$ can never happen

segment which ends first $\rightarrow$ discard intersection

//



//

//



//

```
vector < pair < int, int > > segments(n);  
sort (segments in ascending order of end Time)
```

last_segment $\rightarrow \{-inf, -inf\}$

```
for ( int i = 0 ; i < n ; i++ ) {
```

```
    if segments[i].endTime > last_segment.startTime
```

```
        ans++
```

```
        last_segment = segments[i]
```

}



Activity Selection Problem

If we want to choose the very first activity, which one should it be?

Choosing that activity which has the lowest ending time allows us to choose more activities in future.

We know this works for first activity, we can argue the same for 2nd activity, the 3rd activity and so on....

Let's just sort all the activities by their ending time and then pick them up in order.



Fractional Knapsack Problem

Given two arrays named value and weight of size N each, where $\text{value}[i]$ represents the value you get if you choose i^{th} item, and $\text{weight}[i]$ represents the weight of the i^{th} item.

You want to maximize the value such that the sum of weights of the chosen items is $\leq W$. You are allowed to choose a fraction of an item.

V

10	2	1	3	4	12	6
----	---	---	---	---	----	---

W

1	2	1	2	1	1	3
---	---	---	---	---	---	---

$$W = 5$$

Claim $\rightarrow$ pick the maximum weight of every item possible such that total weight $\leq W$ and items are sorted in descending order of $\frac{\text{value}(i)}{\text{weight}(i)}$

$$\underline{\underline{W = 1}}$$

example :

V	[10 2 1 3 4 12 6]						
W	[1 2 1 2 1 1 3]						
$\frac{V}{W}$	10	1	1	1.5	4	12	2
	—	—	—			=	

$$12 \quad 10 \quad 4 \quad 6 \quad \frac{\text{rem-w}}{\text{item-w}} \times \text{item-value} = \frac{1}{2} \times 3 = \underline{\underline{1.5}}$$

$$\begin{aligned} \text{item}_1 &\rightarrow v = 10, & w = 5 \\ \text{item}_2 &\rightarrow v = 6, & w = 1 \end{aligned}$$

$$\begin{aligned} \text{item}_{11} &\Rightarrow v = 2, w = 1 \\ \text{item}_{12} &\Rightarrow v = 2, w = 1 \\ \text{item}_{13} &\Rightarrow v = 2, w = 1 \\ \text{item}_{14} &\Rightarrow v = 2, w = 1 \\ \text{item}_{15} &\Rightarrow v = 2, w = 1 \\ \text{item}_2 &\Rightarrow v = 6, w = 1 \end{aligned}$$

→ bring all to weight,

$$\text{item} = v = v, w = w$$

$$\text{w items} \Rightarrow v = \frac{v}{w}, w = 1$$

Fractional Knapsack Problem



Which item should we choose first? The one with maximum ratio of $\text{value}[i]/\text{weight}[i]$ because that would give us the maximum value of a unit weight.

Which item should we choose next? Same idea

Sort all the items in decreasing order of $\text{value}[i]/\text{weight}[i]$ and keep taking the maximum amount of weight possible from it ensuring total weight $\leq W$

Kadane's Algorithm



$$N \leq 10^5$$

$$-10^9 \leq a[i] \leq 10^9$$

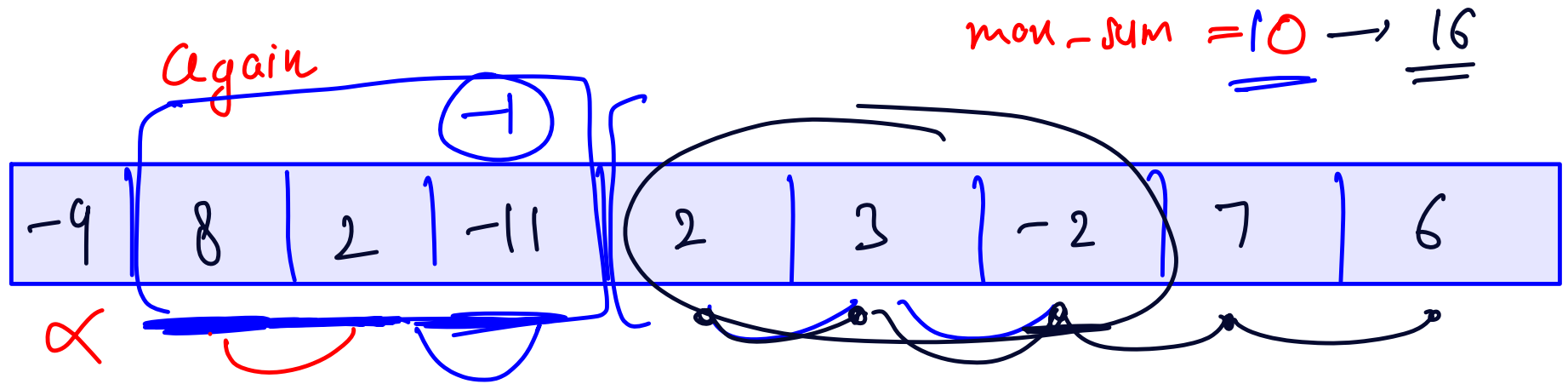
Given an array of N integers, find the maximum possible sum of a subarray.

Example:

$A = [1, 2, -9, 2, 3, -1, 4]$

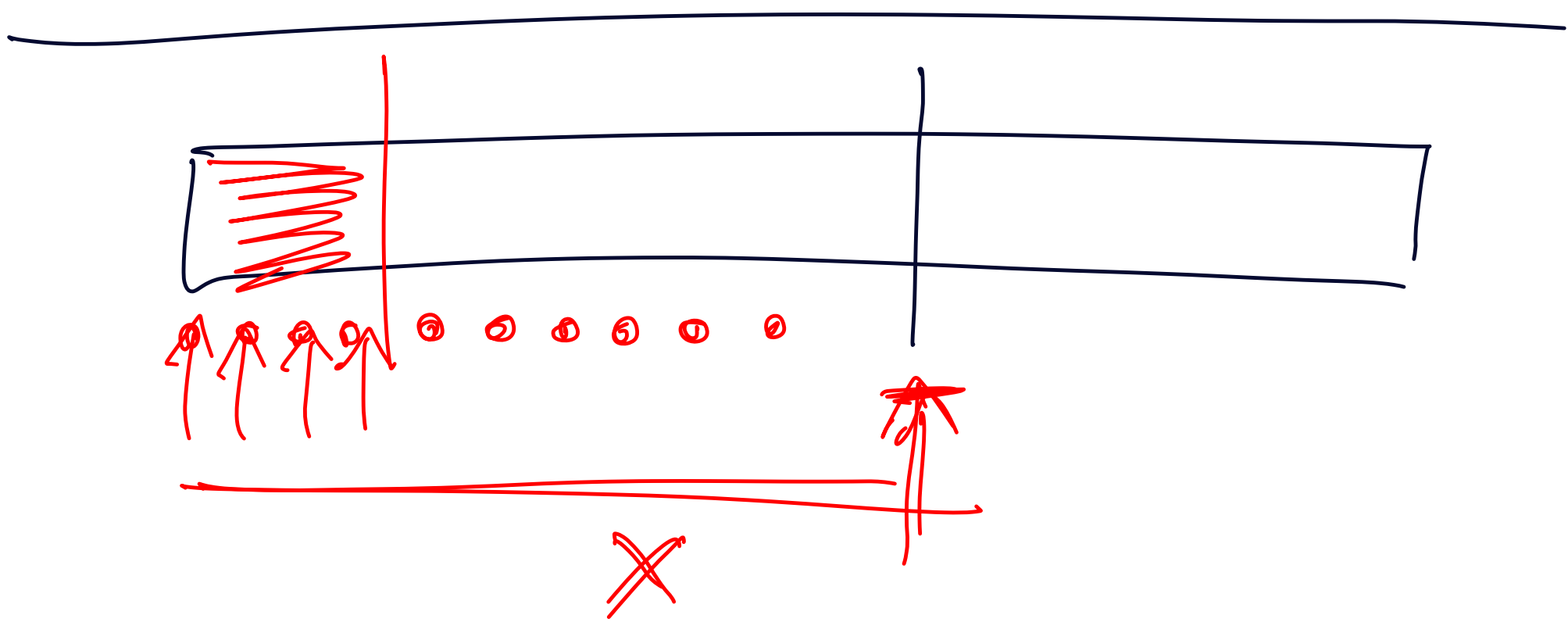
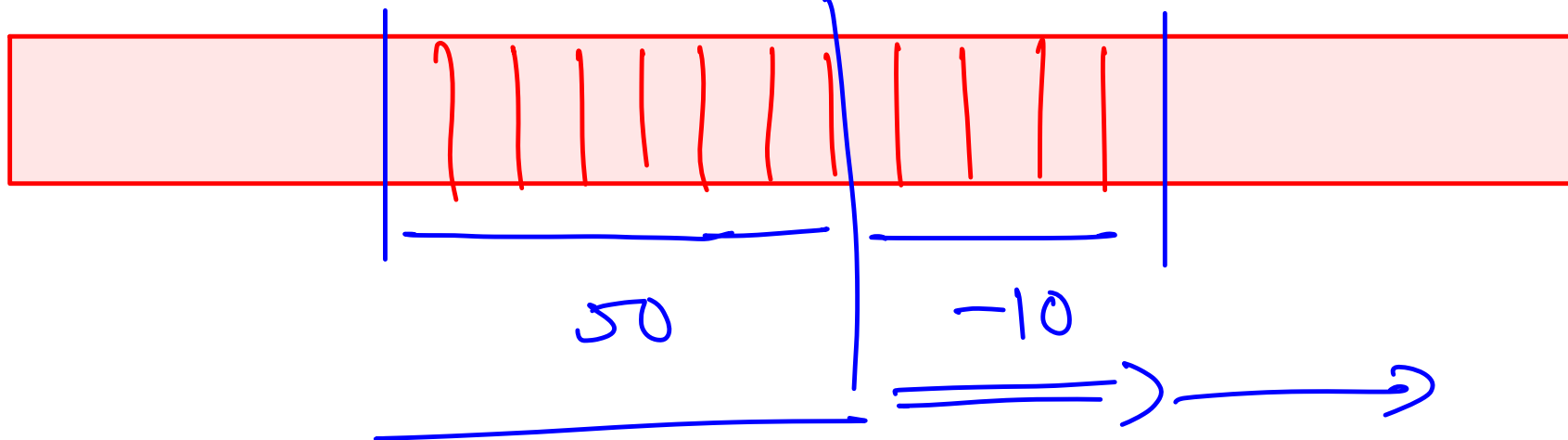
Best subarray = $[2, 3, -1, 4]$, Answer = 8

Claim: pick earliest tr and if on adding next_element sum becomes < 0 then start



mon_sum subarray always ends and starts with a positive element

max_sum subarray has every prefix containing a tr sum and every suffix containing a tr sum



① Kadane's Algorithm

```
int max_sum = -inf;
```

```
int sum = 0
```

```
for (int i = 0; i < n; i++) {
```

```
    sum += arr[i];
```

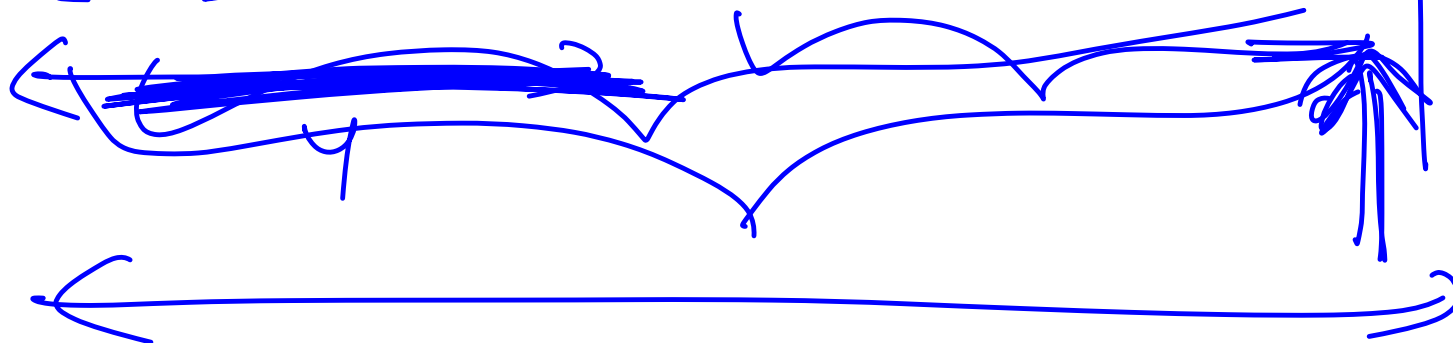
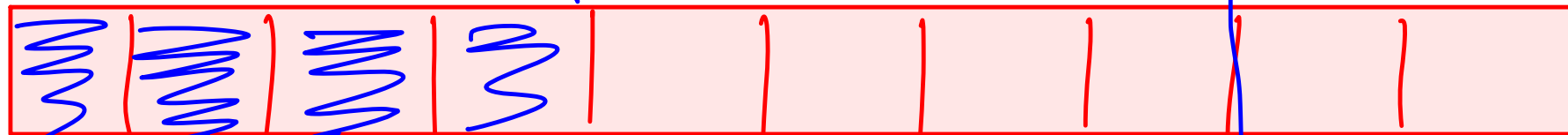
```
    max_sum = max(max_sum, sum)
```

```
    if (sum < 0)
```

```
        sum = 0
```

```
}
```

②



$x - y$





Kadane's Algorithm

Is it every feasible to include a negative integer in the subarray?

When should we include a negative integer?

How can we simulate the process greedily?

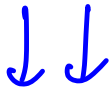
Try to include elements in current subarray until the subarray sum > 0

Job Sequencing Problem

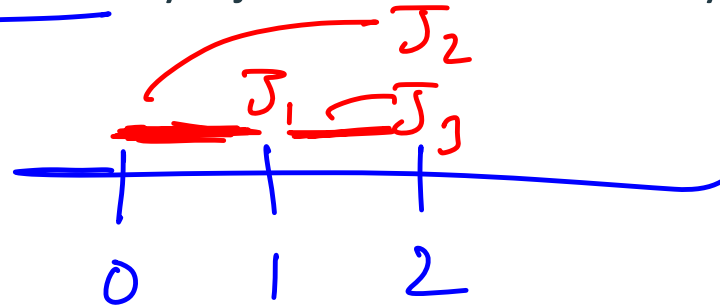


Given an array of jobs where every job has a deadline and associated profit, if the job is finished before the deadline. It is also given that every job takes a single unit of time. Maximize the profit if only 1 job can be done at any time.

Example:



Time = [1, 2, 2], Profit = [10, 20, 30]



The best combination includes picking up 2nd and 3rd one -> Profit = 50

Claim 1 : sort jobs by profit in descending order and pick each of them from left to right if possible and place them in the last available slot upto its deadline

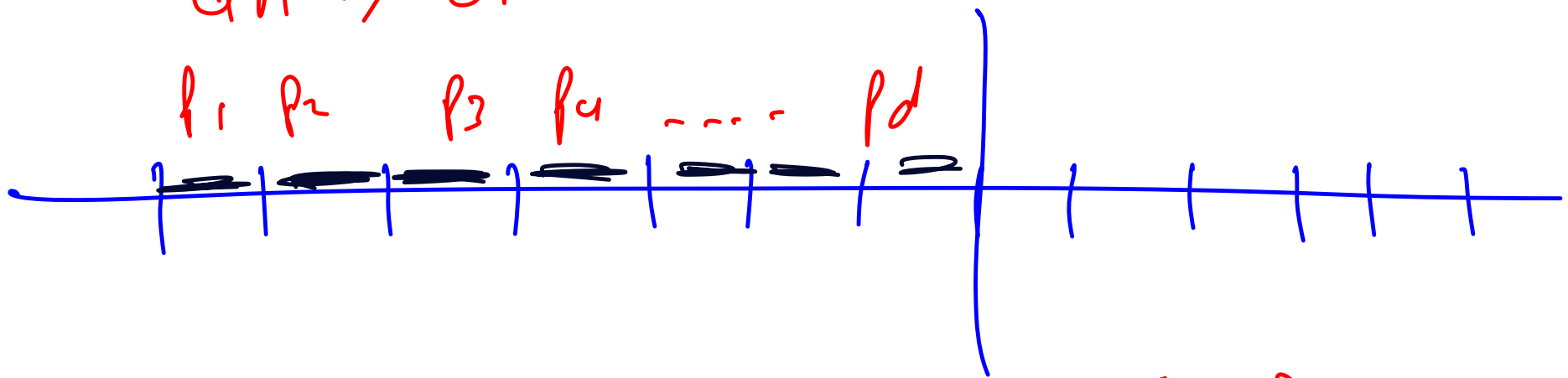
- ① which job to choose that ✓
- ② where to place it ✓

① Choose job with max profit first

$GA = \text{max-profit job} + \text{some other jobs}$
 $OA = \text{! max-profit job and some jobs}$

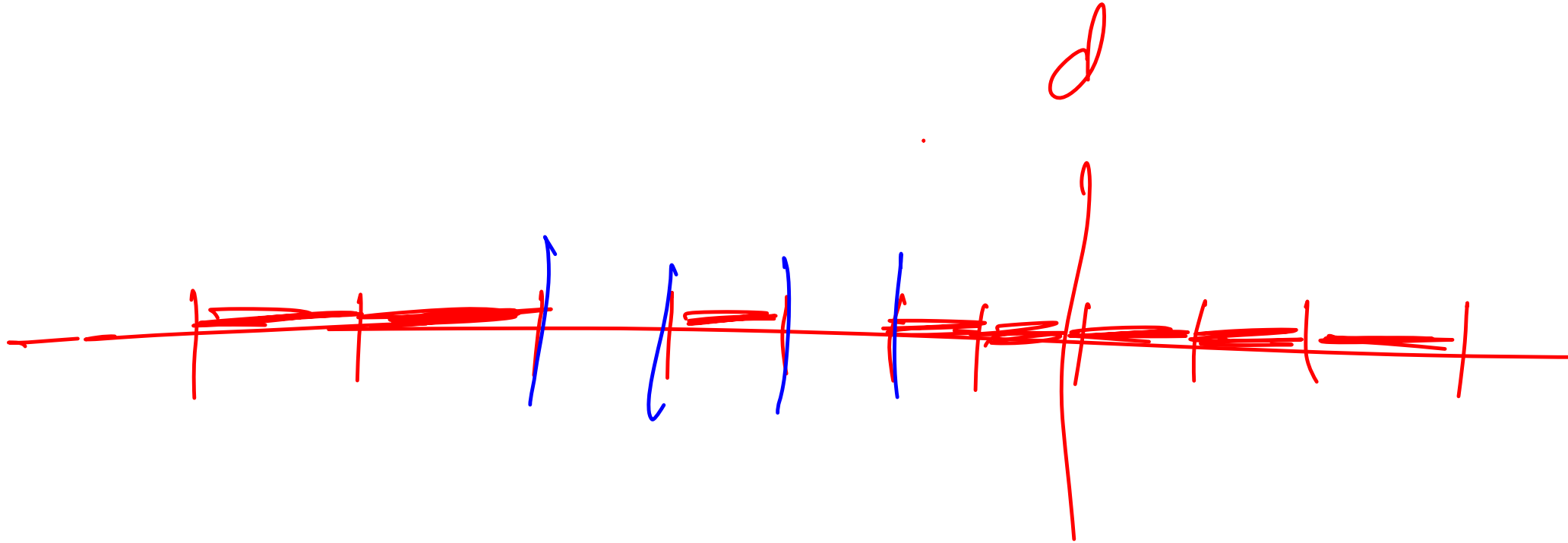
$$GA \geq OA$$

$d = \text{max-profit-job}$

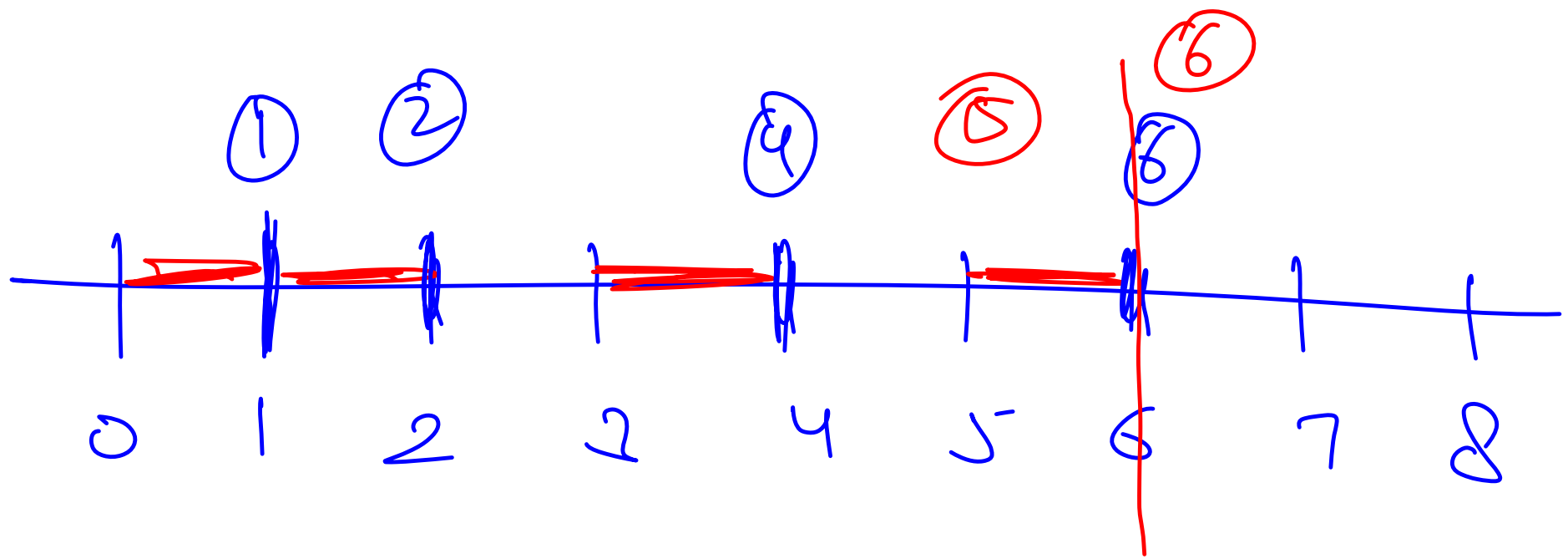


$$p_1, p_2, p_3, \dots, p_d \leq p_{\text{max}}$$

② place the job in last available slot



Sort jobs in descending order of profit



multiset $\rightarrow$ 1, 2, 4, 6
selected

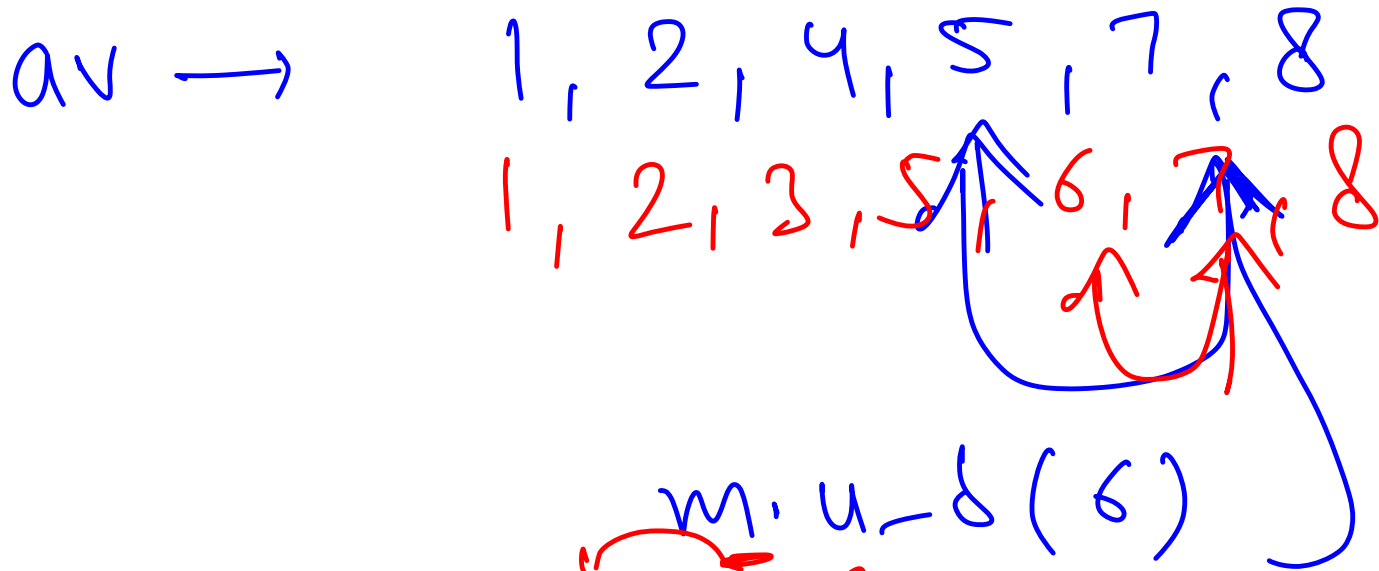
1, 2, 4, 6 $\rightarrow$ 5

1, 2, 4, 5, 6 $\rightarrow$ 3

1, 3, 4, 5, 6 $\rightarrow$ 2

multiset-avg
3, 5, 6, 7, 8

max-upper-bound (6) - 1



$$\underline{\underline{d = 6}}$$

prev (multiset_av.upper-bound(d))

Maximum Perimeter Triangle



Given a list of N positive integers, pick up 3 of them which form a valid triangle with the maximum perimeter possible.

Example:

$A = [2, 3, 15, 5, 1, 7]$

Best choice $\rightarrow [3, 5, 7]$ forms a valid triangle with perimeter = 15



Maximum Perimeter Triangle



What if we fix the longest side of the triangle?

Then the other 2 sides must add up to a number $\geq$ longest side

If C is the longest side, $A + B$ must be $\geq C$

What should be the optimal values for A and B ?

How can we find the best triangle for every possible C ?

Advantages of Greedy Solutions



- Being able to think about a greedy solution prevents us from trying out all the possibilities, hence saving a lot of runtime
- Greedy solutions are usually very easy to implement, mostly involve some form of sorting.



Disadvantages of Greedy Solutions

- Greedy solutions lack global awareness
- It is difficult to reason whether or not a greedy solution is correct
- Trying to disprove a greedy solution usually involves lot of hit and trial
- Doesn't try out all the possibilities

Problem Links



- Activity Selection Problem: <https://cses.fi/problemset/task/1629>